Reviewer Pack — Coxeter Group Representation Invariant vs DPLL Search Tree W…

Entry 8aa9e8182cf1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 12:30:10 UTC

Coxeter Group Representation Invariant vs DPLL Search Tree Width

Entry ID: `8aa9e8182cf1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 12:30:10 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Theory **Field B** (complexity object): DPLL search tree complexity

Statement:

> {'quantitative_relation': 'The width of the DPLL search tree for a CNF formula is bounded by the minimal number of reflections required to express the corresponding representation in the Coxeter group, i.e., E[width(T)] ≤ Θ(min_reflections(r))', 'falsifier_friendly_formulation': 'If there exists a CNF formula whose DPLL search tree width exceeds the minimal number of reflections required to represent it in any Coxeter group, then the conjecture is false.'}

Rationale (proposer's reasoning):

> {'invariant_exposure': 'Coxeter groups encode symmetries and geometric structures, which can be used to capture combinatorial properties of boolean functions. The invariant may expose the underlying complexity of DPLL search trees by connecting them to well-studied algebraic structures.', 'complexity_structure_linkage': 'The relationship between Coxeter group representations and DPLL search tree width could potentially reveal new structural insights into the complexity of satisfiability problems.'}

Taxonomy category: `COXETER_GROUP_INVARIANT` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `271e644c34457249`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between the minimal number of Coxeter group reflections and the DPLL search tree width is ≥ 0.8, and the mean difference in width between the two measures across at least 30 seeds is ≤ 3 units.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def dpll_width(cnf):
        variables = set(abs(lit) for lit in sum(cnf, []))

        def solve(clauses, assignment={}):
            if not clauses:
                return 0
            unit_clauses = [c[0] for c in clauses if len(c) == 1]
            pure_lits = {}
            for lit in variables:
                pos_count = sum(1 for c in clauses if lit in c)
                neg_count = sum(1 for c in clauses if -lit in c)
                if pos_count == 0:
                    pure_lits[lit] = True
                elif neg_count == 0:
                    pure_lits[-lit] = True

            if unit_clauses:
                literal = unit_clauses[0]
                new_assignment = assignment.copy()
                new_assignment[literal] = True
```

```
                return solve([c for c in clauses if literal not in c and -literal not in c], new_a

            if pure_lits:
                literal = next(lit for lit, val in pure_lits.items() if val)
                new_assignment = assignment.copy()
                new_assignment[literal] = True
                return solve([c for c in clauses if literal not in c and -literal not in c], new_a

            literal = variables.pop()
            width_positive = 1 + solve(clauses, {**assignment, literal: True})
            width_negative = 1 + solve(clauses, {**assignment, literal: False})
            return max(width_positive, width_negative)

        return solve(cnf)

    def min_reflections(cnf):
        # Placeholder for the actual computation of minimal reflections
        # This is a dummy implementation and should be replaced with the actual logic
        return len(cnf) // 2

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    width = dpll_width(cnf)
    min_reflect = min_reflections(cnf)

    return {
        "metric_name": "DPLL Width vs Min Reflections",
        "metric_value": abs(width - min_reflect),
        "instances_tested": 1,
        "conjecture_holds": abs(width - min_reflect) <= 3,
        "counterexample": "" if abs(width - min_reflect) <= 3 else f"Width: {width}, Min Reflectio
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]  # Default to first

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_hold
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 87, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 70, in run_trial
    width = dpll_width(cnf)
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 61, in dpll_width
    return solve(cnf)
           ^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 54, in solve
    return solve([c for c in clauses if literal not in c and -literal not in c], new_assignment)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 54, in solve
    return solve([c for c in clauses if literal not in c and -literal not in c], new_assignment)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 54, in solve
    return solve([c for c in clauses if literal not in c and -literal not in c], new_assignment)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 993 more times]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44629c87.py", line 37, in solve
    pos_count = sum(1 for c in clauses if lit in c)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide a Pearson correlation coefficient or mean difference in width. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15601

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
2	preregistration	ollama_remote	glm4:latest	0	0	9486
3	novelty	ollama_remote	glm4:latest	0	0	8826
4	novelty	ollama_remote	glm4:latest	0	0	8509
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30068
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11623
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10881
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10655
9	judge	ollama_remote	glm4:latest	0	0	11979

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 117627 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8aa9e8182cf1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8aa9e8182cf1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8aa9e8182cf1.tar.gz` (if generated)